

Grant Seeker AI - Architecture & Multi-Agent Technical Summary

Generated: 2026-03-18

1) System Architecture Summary

Grant Seeker AI is implemented as a hybrid multi-agent architecture using Google ADK. The runtime combines sequential orchestration for planning/discovery and concurrent extraction for throughput. The primary orchestrator is GrantSeekerWorkflow in backend/adk_agent.py. User interactions come through Streamlit pages, then route into asynchronous ADK runs.

- Frontend entry for discovery: frontend/search_grants.py -> execute_grant_workflow(...).
- Backend orchestrator: backend/adk_agent.py -> GrantSeekerWorkflow.run(...).
- Writer workflow: frontend/proposal_builder.py -> backend/writer_agent.py -> draft_proposal_section(...).
- Per-agent sessions are managed with InMemorySessionService and unique session IDs.

2) End-to-End Runtime Pipeline

Pipeline implemented in GrantSeekerWorkflow.run(query):

- Phase 0 - Query Generation Agent transforms user natural language into targeted Canadian search queries.
- Phase 1 - Discovery search from Tavily/Google CSE returns candidates; Finder Agent selects top 5-7 leads.
- Phase 2 - Parallel extraction invokes Extractor Agent across lead URLs with asyncio Semaphore(3).
- Post-processing filters expired/USA/low-quality results, computes fit scores, validates viability, assigns IDs.
- Optional Phase 3 - Writer Agent drafts proposal text from selected grant + user project narrative.

3) Multi-Agent Areas (Technical Inventory)

All technically verified multi-agent zones in the current codebase:

- A. Query Strategy Agent (backend/adk_agent.py:create_query_agent): ADK LlmAgent named QueryGenerator. Input: free-text mission/project description. Output: single optimized search query string with Canada-specific constraints.
- B. Discovery Ranking Agent (backend/adk_agent.py:create_finder_agent): ADK LlmAgent named GrantFinder. Input: JSON search results. Output schema: DiscoveredLeadsReport (list of DiscoveredLead entries). This is an agent-based filtering/ranking stage.
- C. Parallel Extraction Agent Pool (backend/adk_agent.py:create_extractor_agent + run): ADK LlmAgent named GrantExtractor with response schema GrantData. Multiple agent calls execute in parallel via asyncio.gather, bounded by MAX_CONCURRENT_EXTRACTIONS=3 using Semaphore.
- D. Iterative Multi-Attempt Orchestration (backend/adk_agent.py:run_with_minimum_results): A supervisory loop that repeatedly runs the full agent workflow with query variants until min_results or max attempts (5). This creates multi-round, multi-agent execution batches.
- E. Proposal Writer Agent (backend/writer_agent.py:writer_agent): Independent ADK LlmAgent named GrantWriter. Triggered from frontend/proposal_builder.py. Input: project narrative + selected grant fields; Output: markdown draft sections.
- F. Cross-agent typed contract layer: Pydantic schemas DiscoveredLeadsReport/DiscoveredLead/GrantData enforce strict, typed handoff between agent stages and UI.

4) Orchestration, Concurrency, and State

- Runner orchestration uses google.adk.runners.Runner with run_async event streams per stage.
- Session isolation: unique UUID-based session IDs created for query generation and each extraction.
- Concurrency control: asyncio.Semaphore(MAX_CONCURRENT_EXTRACTIONS) prevents uncontrolled parallel calls.
- Fan-out/fan-in pattern: list[lead] -> parallel extraction tasks -> flatten list[list[grant]] -> ranking/filtering.
- Caching in orchestrator reduces repeated search/extraction calls (24h TTL, file-based JSON cache).

5) Multi-Agent Data Contracts

Core structured contracts used by agents:

- DiscoveredLead: url, source, title.

Grant Seeker AI - Architecture & Multi-Agent Technical Summary

Generated: 2026-03-18

- DiscoveredLeadsReport: discovered_leads: list[DiscoveredLead].
 - GrantData: title, funder, deadline, amount, description, detailed_overview, tags, eligibility, url, application_requirements, funding_nature, geography, fit_score, founder_demographics.
- Typed response schemas are configured in Gemini generation_config with response_mime_type='application/json' and response_schema set to Pydantic models.

6) Frontend Integration Points Using Multi-Agent Backend

- Search workflow (frontend/search_grants.py): execute_grant_workflow() calls workflow.run() or workflow.run_with_minimum_results() depending on min result target and active filters.
- Proposal builder (frontend/proposal_builder.py): generate_proposal_with_agent() imports writer_agent and calls draft_proposal_section(), which runs ADK writer agent through a dedicated Runner.
- Grant details export (frontend/grant_details.py): not an agent stage itself; consumes already extracted agent output and serializes to PDF.

7) Guardrails and Filtering Around Agent Output

- Expired grant detection via deadline parsing and explicit 'expired' checks.
- USA grant exclusion via geography/state-name screening for Canada-only scope.
- Data viability filter rejects low-information outputs (prevents 'Untitled Grant' display).
- Fit scoring ranks results and adaptive threshold keeps top relevance while preserving minimum count.

8) Reliability and Test Coverage Relevant to Multi-Agent

- tests/test_iterative_search.py validates iterative multi-attempt behavior and minimum target logic.
- tests/test_advanced.py validates robustness under ambiguous/specific/bilingual query scenarios.
- tests/test_filters.py validates post-agent filtering relevance and data completeness rules.

9) Technical Notes and Observed Design Decisions

- The architecture is hybrid, not a free-form autonomous swarm; stages are explicitly orchestrated.
- Parallelism exists specifically in extractor stage; query/finder/writer are single-agent per invocation.
- Iterative mode composes multiple full workflows, effectively multiplying agent invocations by attempt count.
- Content extraction uses deterministic fallback strategies (Tavily/Google/PDF/HTML) before LLM extraction.

10) Concise Multi-Agent Map

User Query -> QueryGenerator Agent -> Search Provider -> GrantFinder Agent -> Lead List -> [Parallel GrantExtractor Agent Calls xN (max 3 concurrent)] -> Filtering/Ranking -> UI Results -> (Optional) GrantWriter Agent for proposal drafting.